Comparing REINFORCE and SAC on FetchReach

Steven Chen
CS 5180 – Reinforcement Learning Project — April 14, 2026

Problem: FetchReach MDP

State Space $\mathcal{S}$

The environment provides a dictionary observation. I flatten the following components into a single vector:

$\mathbf{o} \in \mathbb{R}^{10}$	robot joint angles and gripper state (velocity + position)
$\mathbf{g}_d \in \mathbb{R}^3$	desired goal position (x_d, y_d, z_d)
$\mathbf{g}_a \in \mathbb{R}^3$	achieved goal (current gripper position) (x_a, y_a, z_a)

For REINFORCE, I additionally concatenate the relative error $\mathbf{g}_d - \mathbf{g}_a \in \mathbb{R}^3$, giving a **19-dim** state vector. This explicit error signal helps the policy learn directional movement.

Action Space $\mathcal{A}$

4-dimensional continuous actions: $a = [\Delta x, \Delta y, \Delta z, \text{gripper}]$, each component in $[-1, 1]$. The first three components specify Cartesian displacement of the end-effector; the fourth controls gripper actuation (unused in this task).

Reward Function $\mathcal{R}$

The original environment uses a **sparse reward**:

$$r_{\text{sparse}} = \begin{cases} 0 & \text{if } \|\mathbf{g}_a - \mathbf{g}_d\| < 0.05 \text{ (episode ends)} \\ -1 & \text{otherwise} \end{cases}$$

This provides no gradient toward the goal, making exploration extremely difficult for on-policy methods.

I implement a **dense reward wrapper** for both algorithms to enable learning:

$$r_{\text{dense}} = -\texttt{base_scale} \cdot \|\mathbf{g}_a - \mathbf{g}_d\| + \begin{cases} \texttt{success_bonus} & \text{if } \|\mathbf{g}_a - \mathbf{g}_d\| < 0.05 \\ 0 & \text{otherwise} \end{cases}$$

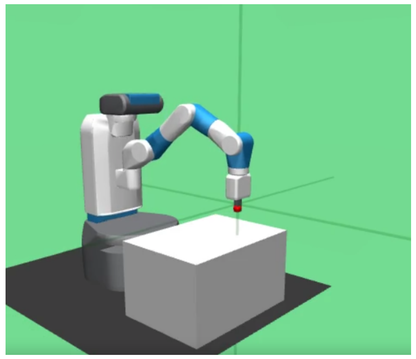
Typical values: $\texttt{base_scale} = 1.0$ (REINFORCE) or 3.0 (SAC), $\texttt{success_bonus} = 50.0$. The episode terminates upon success or after a maximum number of steps

Transition Dynamics $\mathcal{P}$

Deterministic transitions governed by the MuJoCo physics engine. The next state is fully determined by the current state and action.

Why is this challenging?

- Sparse reward: random exploration almost never reaches the goal.
- Continuous control: fine-grained movements required.
- Short horizon: only 50 steps by default.



Algorithm 1: REINFORCE with Baseline

From Sutton & Barto (Chapter 13). On-policy Monte Carlo policy gradient.

Initialize policy π_θ (Gaussian mean & log-std) and value V_ϕ
for each episode **do**
  Collect trajectory $\tau = (s_0, a_0, r_0, \ldots, s_T)$ using π_θ
  Compute discounted returns G_t for all t
  for each step t in τ **do**
    $\delta_t \leftarrow G_t - V_\phi(s_t)$
    $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t|s_t)\, \delta_t$
    $\phi \leftarrow \phi - \beta \nabla_\phi \big(G_t - V_\phi(s_t)\big)^2$
  end for
end for

- Dense reward wrapper (scale=1.0, bonus=50.0).
- Batch update (16 episodes) with return normalisation.
- Gradient clipping (max_norm=0.5), learning rate scheduling.
- Relative goal concatenated to state (+3 dims).
- Hidden layers: 512. Trained for 100k episodes.

Algorithm 2: SAC+HER (still in progress)

Soft Actor-Critic (off-policy) with **Hindsight Experience Replay**.

Init actor π_θ, critics Q_{ϕ_1}, Q_{ϕ_2}, target critics, entropy coeff α
Init replay buffer $\mathcal{B}$
for each episode **do**
  Collect trajectory τ with original goal g
  Store τ in $\mathcal{B}$
  for each transition t in τ **do**
    Sample k future achieved goals g' from τ
    Relabel transition with new goal g' and recompute reward
    Store relabeled transition in $\mathcal{B}$
  end for
  for $u = 1$ to U **do**
    Sample mini-batch (s, a, r, s', d, g) from $\mathcal{B}$
    $a', \log \pi(a'|s') \leftarrow \pi_\theta(s')$
    $y \leftarrow r + \gamma(1-d)\big(\min_{i=1,2} Q_{\phi_i,\text{targ}}(s', a') - \alpha \log \pi(a'|s')\big)$
    Update critics: $\phi_i \leftarrow \phi_i - \lambda_Q \nabla_{\phi_i} \text{MSE}(Q_{\phi_i}(s,a), y)$
    Update actor: $\theta \leftarrow \theta - \lambda_\pi \nabla_\theta \big(\frac{1}{B}\sum \alpha \log \pi(a|s) - \min_i Q_{\phi_i}(s,a)\big)$
    Update α to match target entropy
    Soft update target networks: $\phi_{\text{targ}} \leftarrow \tau\phi + (1-\tau)\phi_{\text{targ}}$
  end for
end for

Implementation Details (from scratch):

- Twin Q-networks, automatic entropy tuning.
- HER with `future` strategy ($k = 4$).
- Dense reward (same as REINFORCE) for fair comparison.
- Hidden layers: 256; batch size 128; $\tau = 0.01$.
- Currently trained for 1000 episodes; **will be extended to 5000 episodes for final results**.
- (have little issue with HER)

Implementation & Hyperparameters

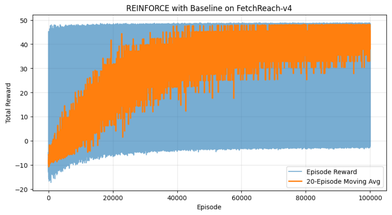
Parameter	REINFORCE	SAC
Optimiser	Adam	Adam
Learning rate	5×10^{-4} (policy)	1×10^{-3} (all)
Hidden layers	512	256
Batch size	16 (episodes)	128 (transitions)
Entropy coef	0.001	automatic
Target update τ	–	0.01
HER k	–	4

Table: Key hyperparameters.

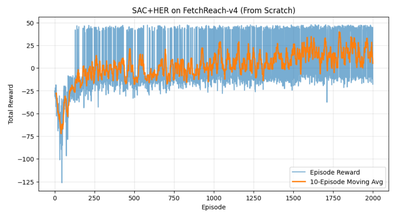
Both algorithms use **dense reward** shaping for a controlled comparison.

Results: Learning Curves

REINFORCE (100k episodes)



SAC (2k episodes, preliminary)



Metric	REINFORCE (100k ep)	SAC+HER (2k ep, prelim)
Success rate	89.6%	$\sim 60\%$
Avg final distance	0.032	0.046
Avg reward	42.8	~ 20
Timesteps to 50% success	$\approx 4,000,000$	$\approx 80,000$
training time	≈ 91 min	≈ 60 min (est.)

Note: SAC+HER performs many more gradient updates per episode, increasing per-step computation.

Analysis & Next Steps

Why the difference?

- **REINFORCE**: On-policy, high variance, needs dense reward to get any signal.
- **SAC**: Off-policy + HER turns failures into learning opportunities. Dense reward can helps but not essential.

Plan to finish:

1. fine tune and continue training SAC to 5000 episodes (expected $> 90\%$ success).
2. Record combined video of multiple successful episodes.
3. Write final report with full analysis.

Conclusion: Classic policy gradient can solve FetchReach with careful engineering, but modern off-policy + HER provides dramatic sample efficiency gains.

See REINFORCE in Action

https://youtu.be/Wq7ZWy14bXg

First 12 second is recorded during training, which are stochastic, which is why it is so erratic. Later video is in eval deterministic environment